

Assembler

Coding for Design I
Demo Day Presentation

Design Challenge

Product designers and engineers frequently encounter difficulties when arranging individual parts with precision, especially when components must align or relate to one another spatially.

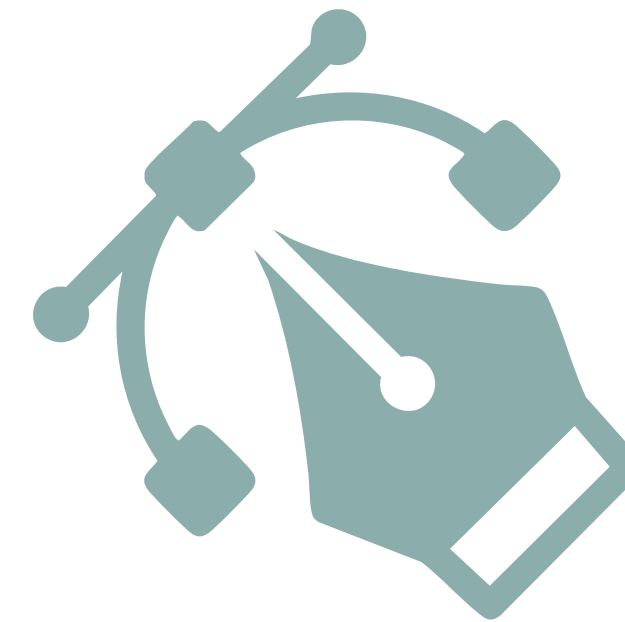
Design Objective

We want to develop a set of components that automates the process of composing these elements efficiently, reduces the time spent aligning parts and ensures precision in assembling these elements.

Target Audience



Product Designers

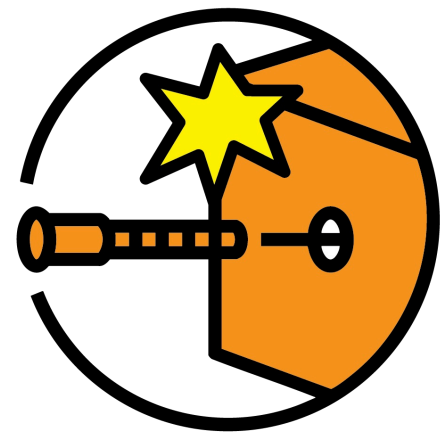


Engineers

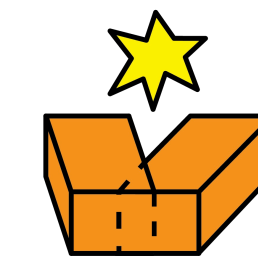
Objective

Main functions and tools we have developed

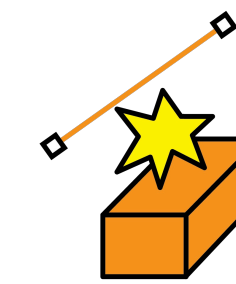
- ✓ Geometry alignment
- ✓ Move Tools
- ✓ Smart measurement
- ☰ Mesh Collision Tool
- ☰ Segment Selection



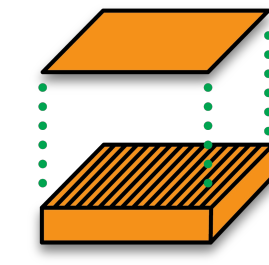
Check Availability



Object
Selection

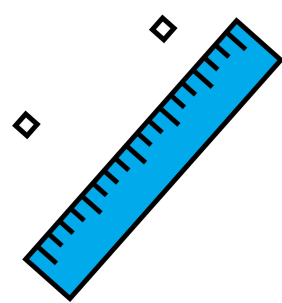


Single Point
Contact

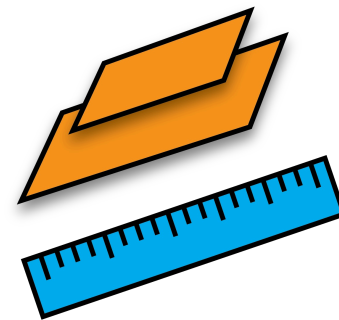


Object
Selection

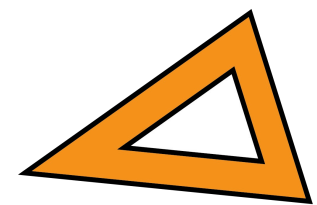
Selection



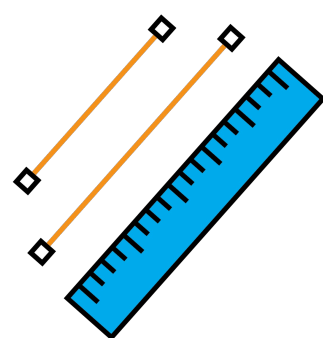
Smart Point



Smart
Surface

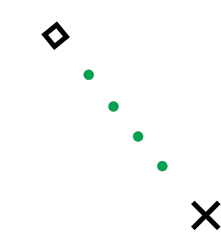


Smart Angle
Line

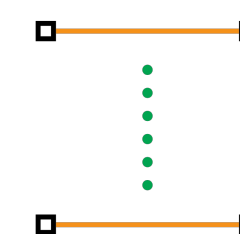


Smart Line

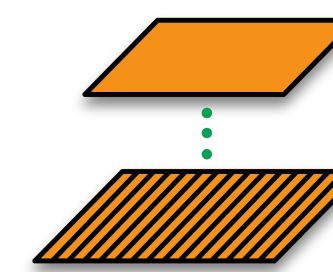
Measure & Model



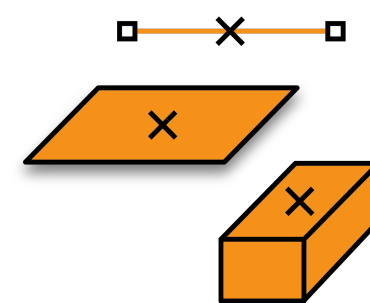
Point
Alignment



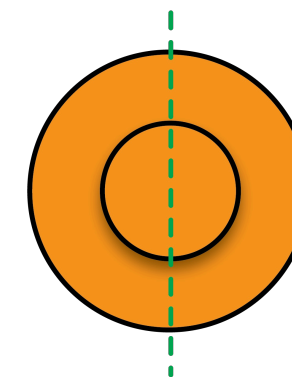
Line
Alignment



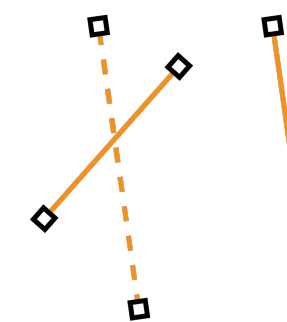
Surface
Alignment



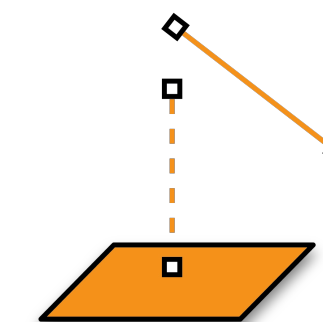
Get MidPoint



Center
Alignment

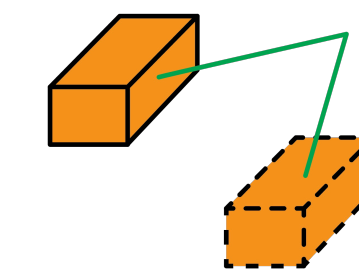


Get Parallel

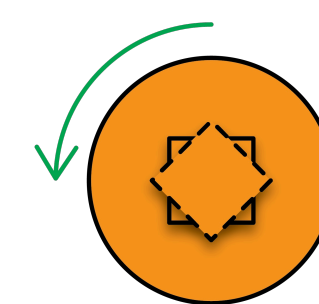


Get Perpendicular

Assembly



Line Mover



Object
Rotator

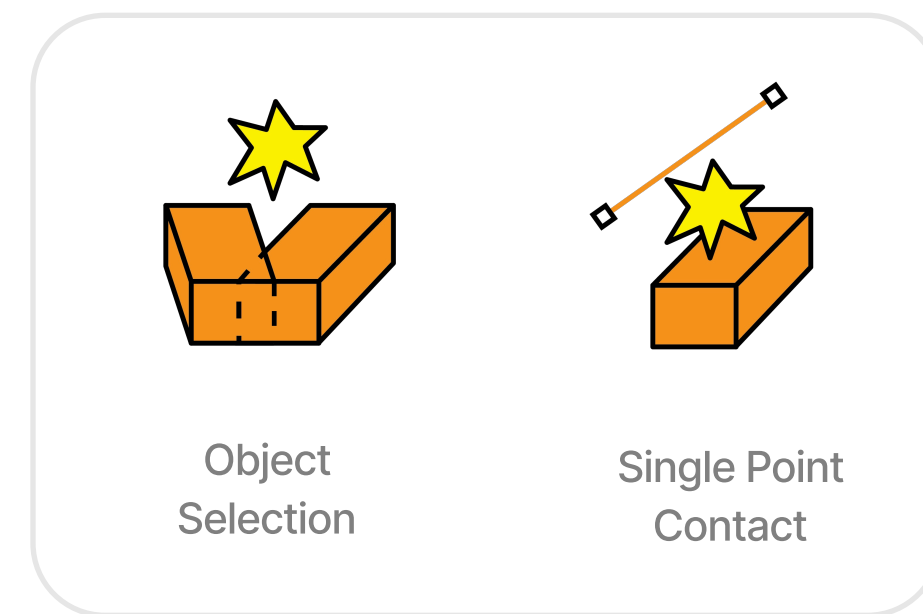
Iterate

Tools

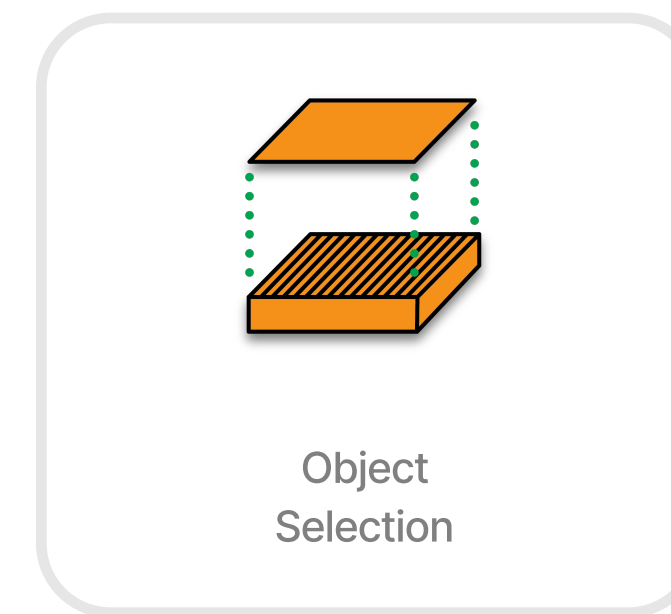


Step 1

Use tools to...



Check Availability



Selection

Tool 1

Object Selection

Extract a point, line or surface of a given subject

INPUT:

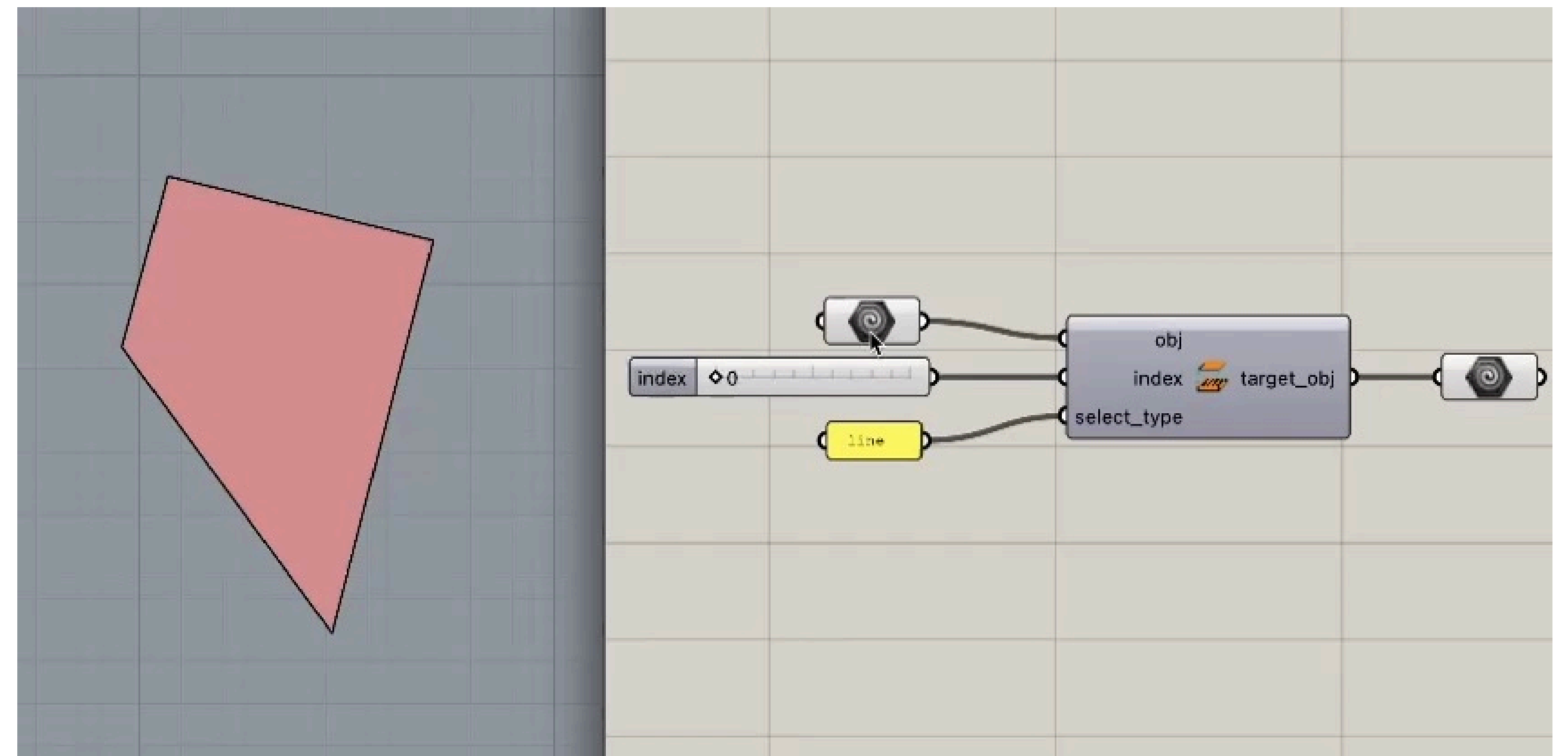
obj: PolyLineCurve, Brep, Mesh

index: number (0-point count)

select_type: string of “point”, “line” or “surface”

OUTPUT:

target_obj: point, line or surface
extracted from given object



Tool 2

Check if two objects have collision or intersect with each other

- Single Point Contact
- Object Selection

Single Point Contact

Check if the g1 single point contact with g2
and return a boolean value

INPUT:

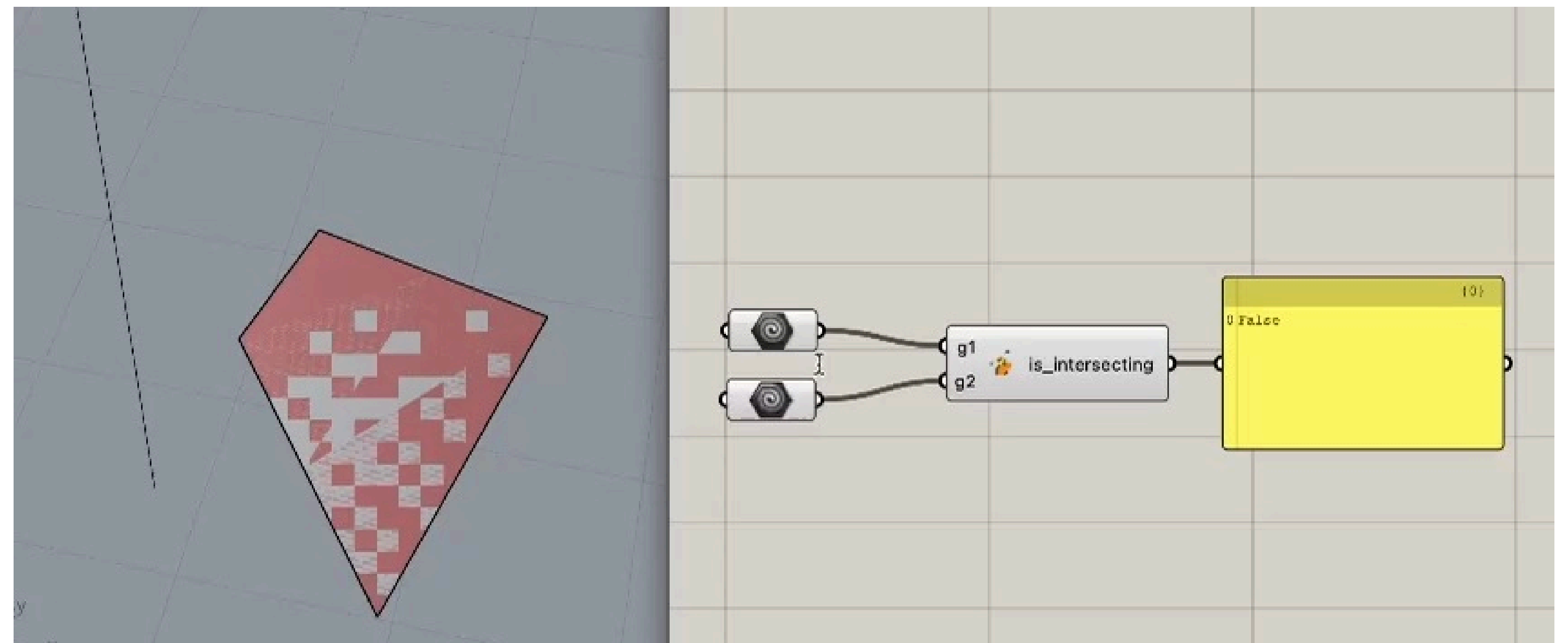
g1: point, line, PolyLine, Curve, Brep

g2: point, line, PolyLine, Curve, Brep

OUTPUT:

is_intersecting:

boolean value - true or false



Object Collision

Check if g1 have intersect with g2 and return a boolean value

INPUT:

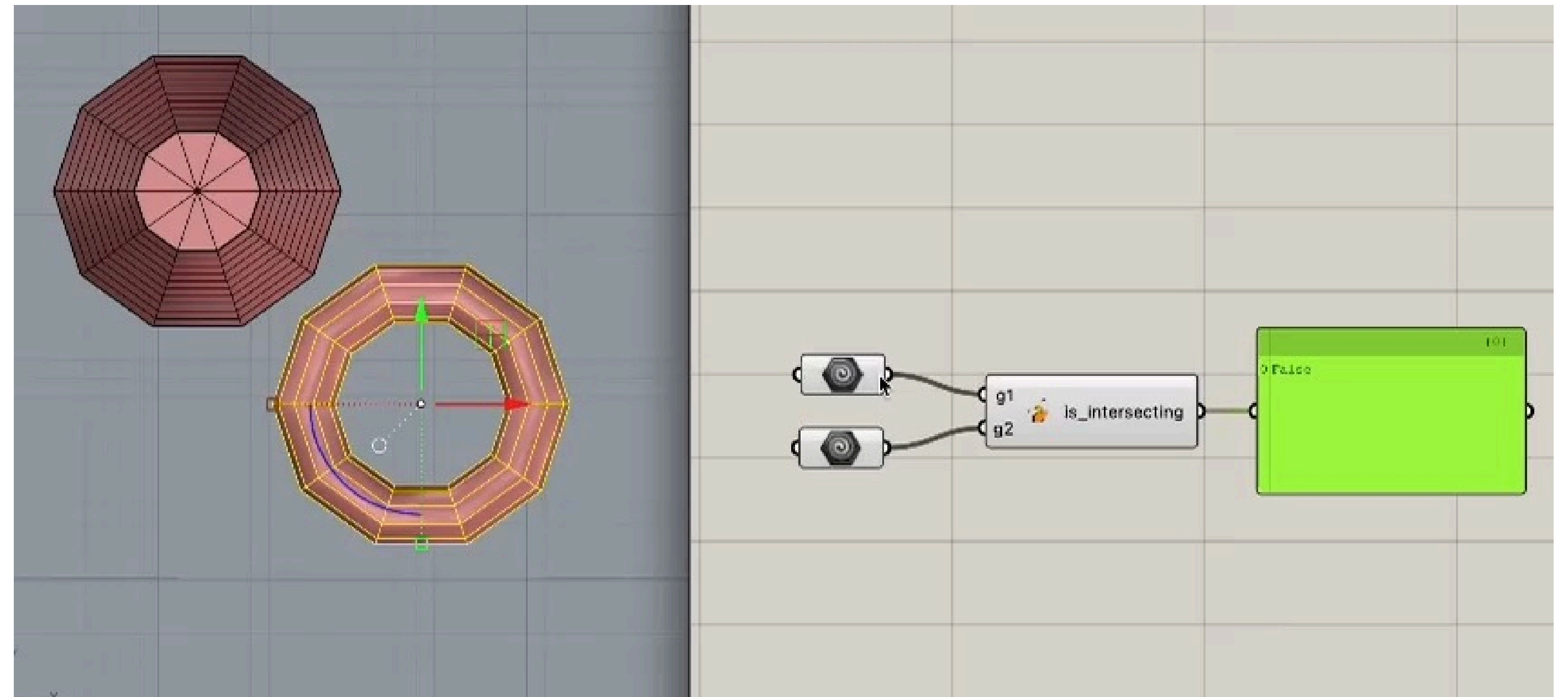
g1: mesh

g2: mesh

OUTPUT:

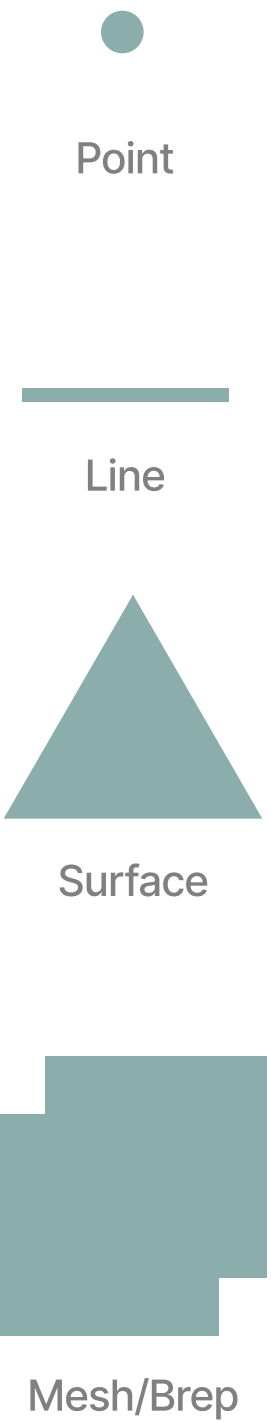
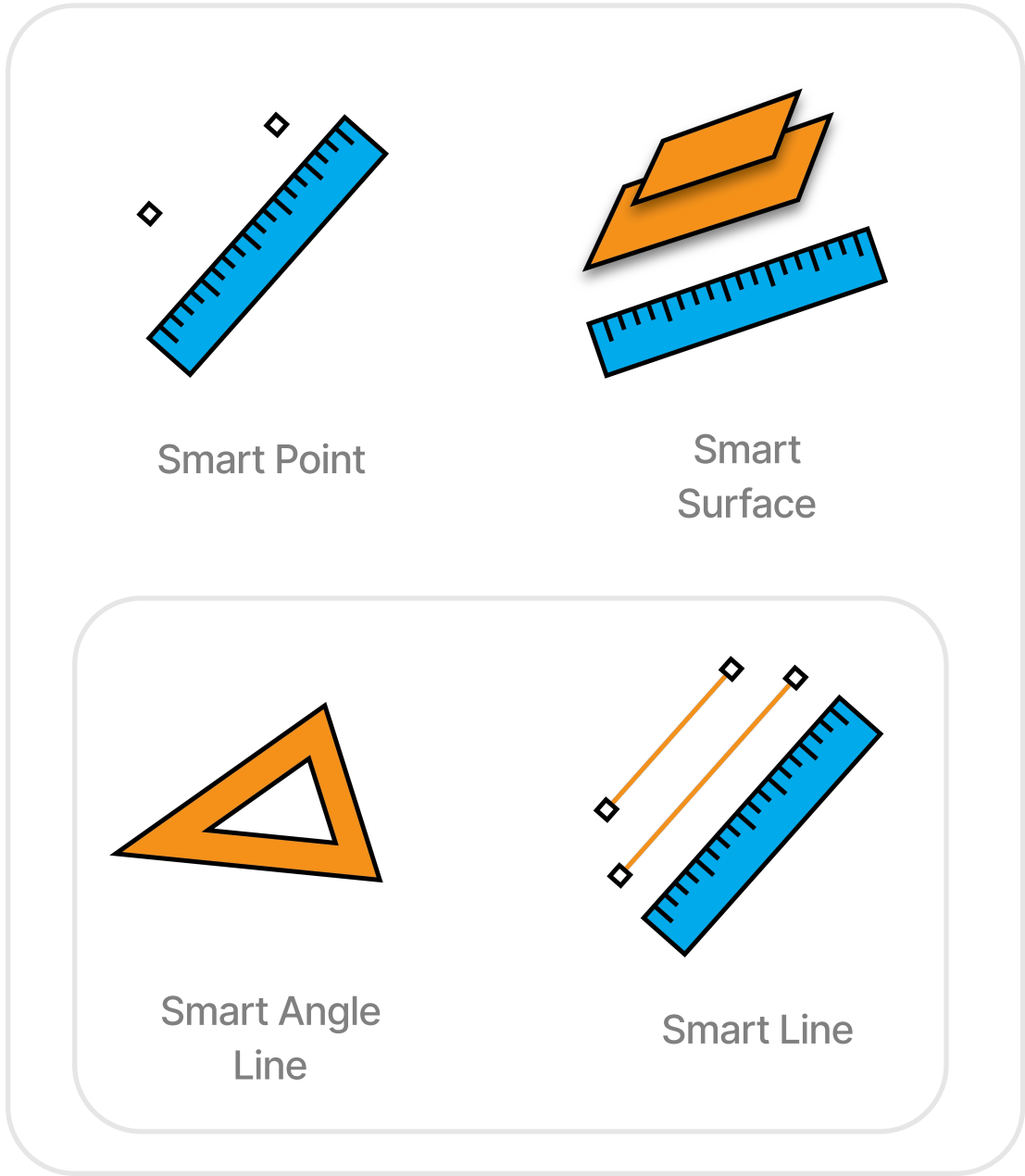
is_intersecting:

boolean value - True or False



Function

Step 2



If users want to generate new subjects...

Smart Measurement

We aim to achieve precise modeling by measuring the distances between points, lines, and surfaces, and generate new ones based on requirements and reference points, lines, and surfaces.

- Smart Point
- Smart Angle Line
- Smart Line
- Smart Surface

Smart Point

Let point 1 to move according to point 2's location with intended distance

INPUT:

point1: target point

point2: reference point

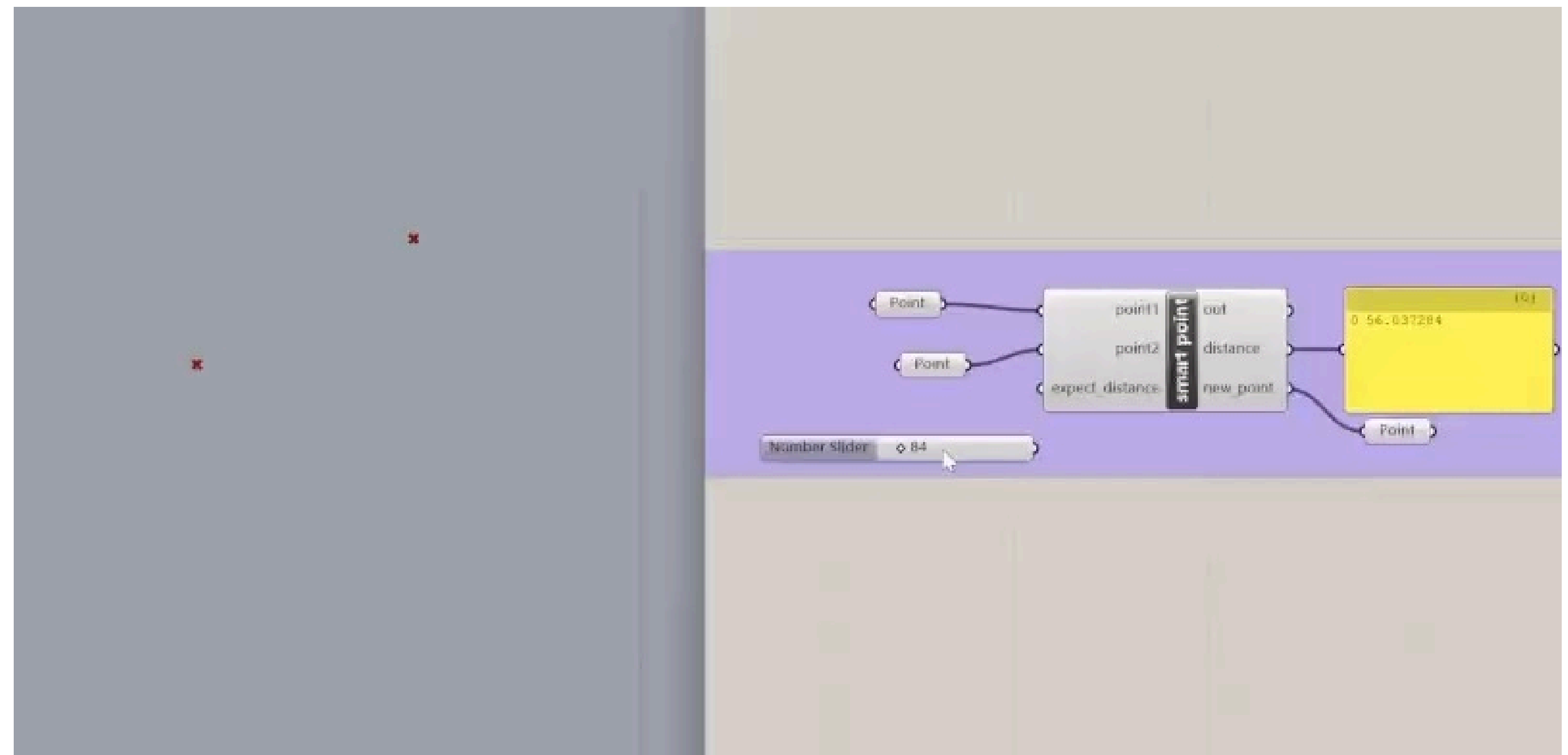
expected_distance:

distance to move according to point2's location

OUTPUT:

distance: $p1 - p2$

point: new point 2



Smart Line

Let line 1 to move according to line 2's location with intended distance

INPUT:

line1: target line

line2: reference line

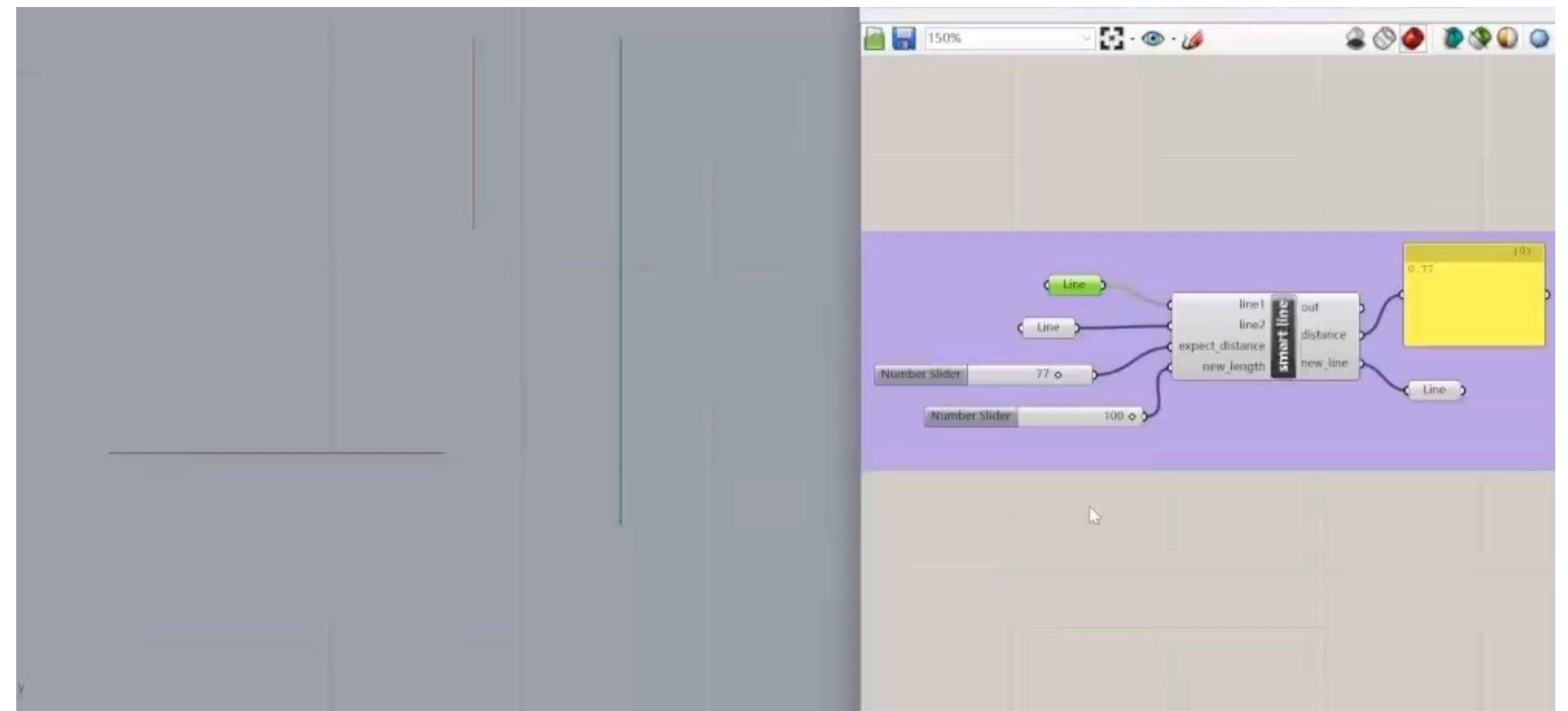
expected_distance:

distance to move according to line2's location

OUTPUT:

distance: $l_1 - l_2$

point: new line 2



Smart Angel Line

Generate a new line based on a reference point or line, using the input distance and angle

INPUT:

point: reference point

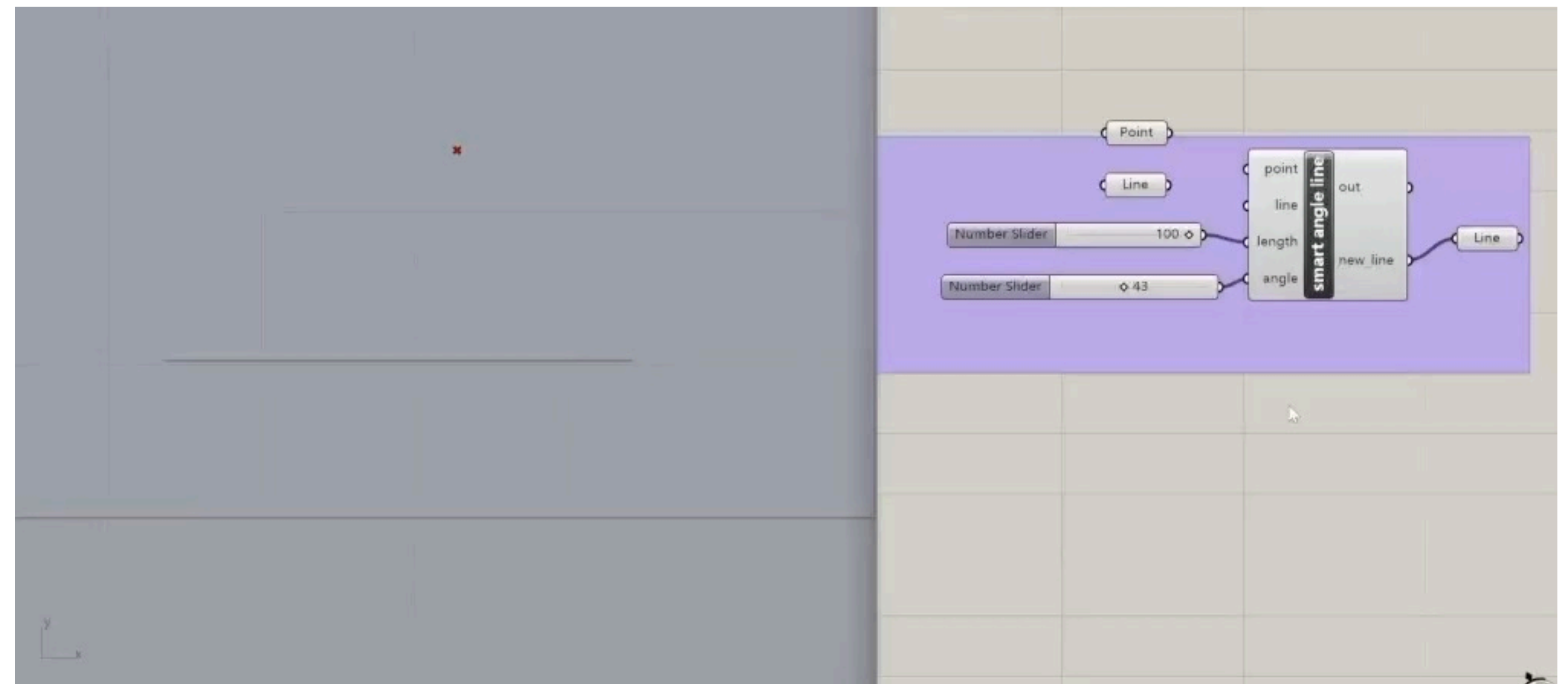
line: reference line

length: the specified input length

angle: the specified input angle

OUTPUT:

new_line: l1 - l2



Smart Surface

Let surface 1 to move according to line 2's location with intended distance

INPUT:

surface1: reference surface

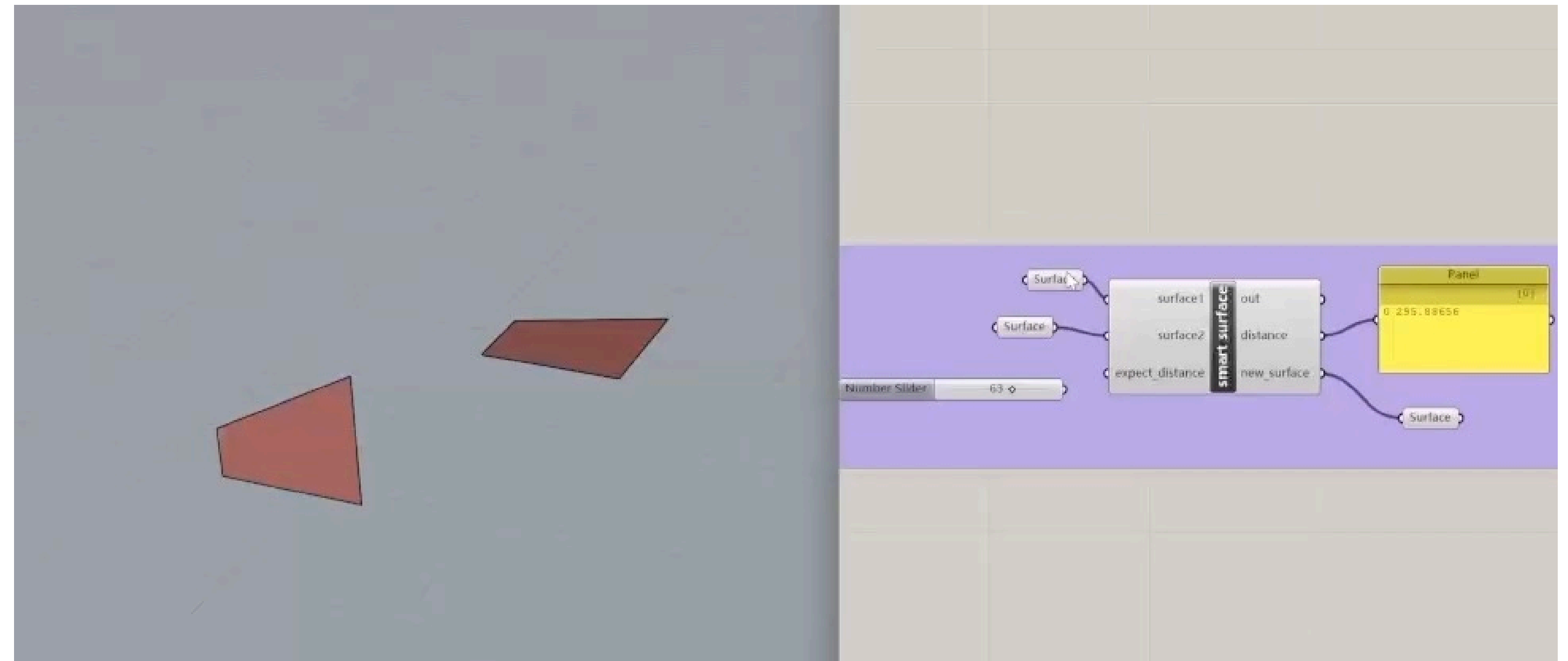
surface2: target surface

expect_distance:
distance to move according to
surface2's location

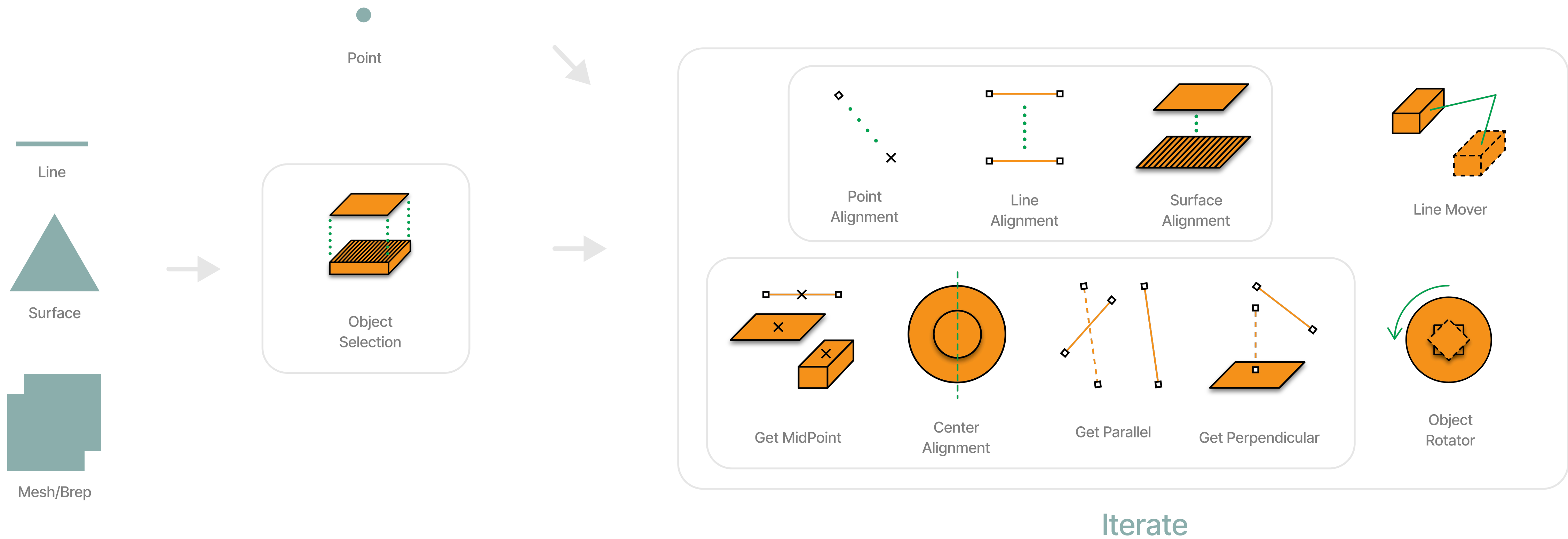
OUTPUT:

distance: $s1 - s2$

new_line: new surface2



Step 2



If users are clear about what they want to iterate...

Geometry Alignment

We want to align two disconnected points, lines, or surfaces within or without a geometry, even if they were created through different logic.

- Point Alignment
- Line Alignment
- Face Alignment
- Center Alignment
- Get MidPoint
- Get Parallel
- Get Perpendicular

Point Alignment

Align Point 1 and Point 2, and obtain the new position of the object

INPUT:

point1: reference point

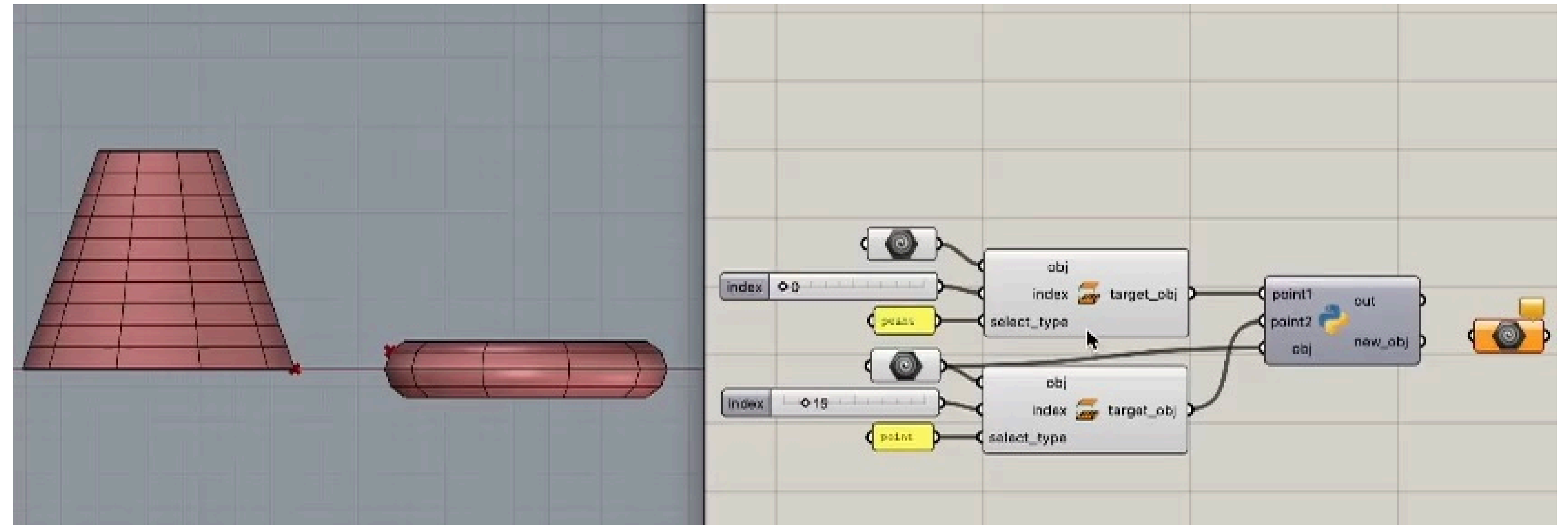
point2: target point

obj: GeometryBase

OUTPUT:

new_obj:

new GeometryBase



Line Alignment

Align Line 1 and Line 2, and move Line 2 along the X-axis, Y-axis, and Z-axis of Line 1

INPUT:

line1: reference lineCurve

line2: target lineCurve

x: number to move along x-axis

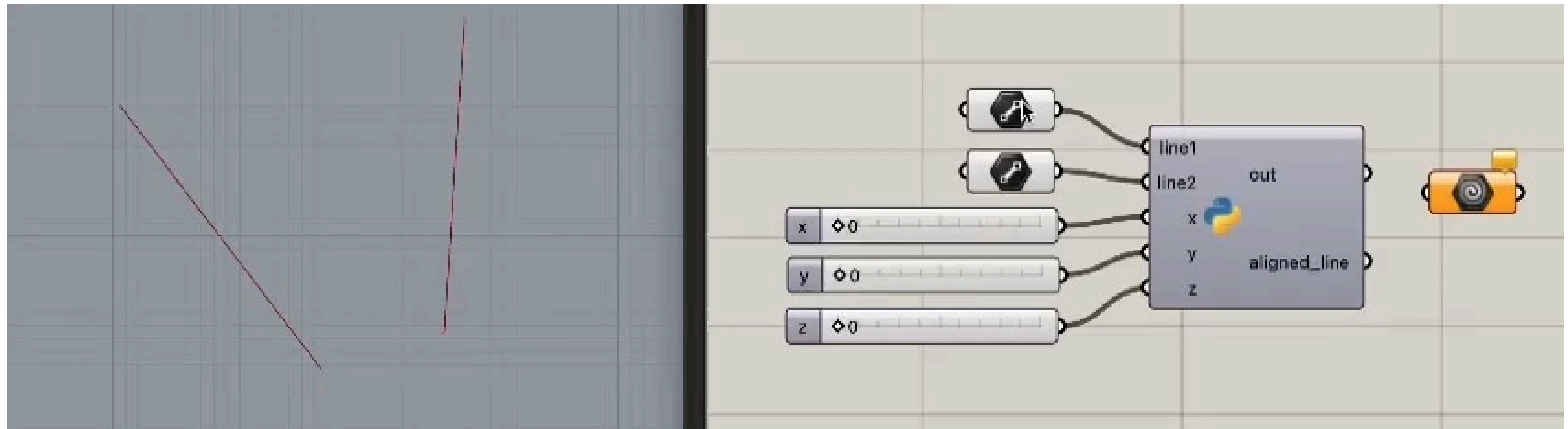
y: number to move along y-axis

z: number to move along z-axis

OUTPUT:

aligned_line

new lineCurve 2



Surface Alignment

Align Surface 1 and Surface 2, and move Surface 2 along the X-axis, Y-axis, and Z-axis of Surface 1

INPUT:

surface1: reference surface

surface2: target surface

x: number to move along x-axis

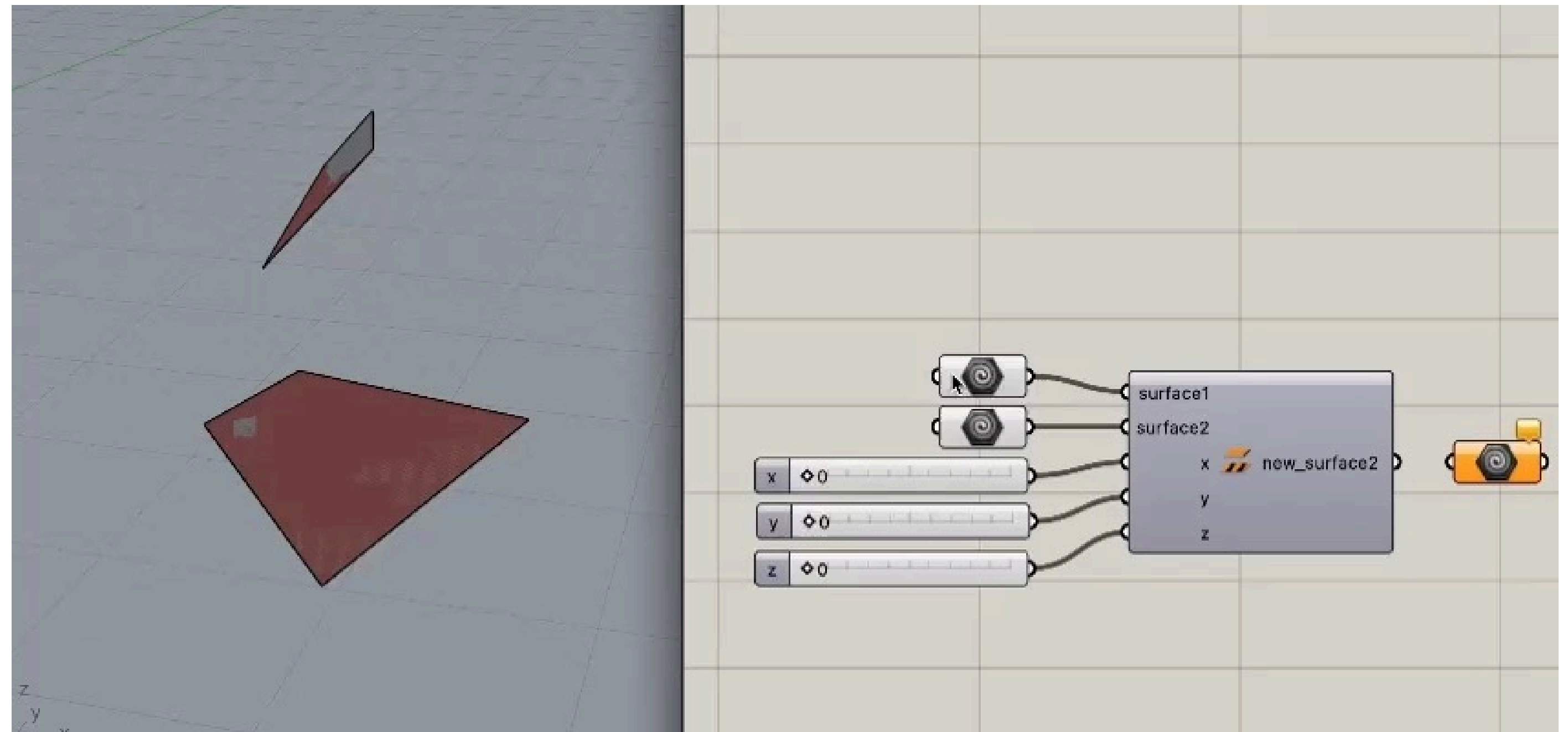
y: number to move along y-axis

z: number to move along z-axis

OUTPUT:

new_surface2:

new surface 2



Center Alignment

Input two objects to align their centers, with the option to adjust their angles

INPUT:

surface1: reference surface

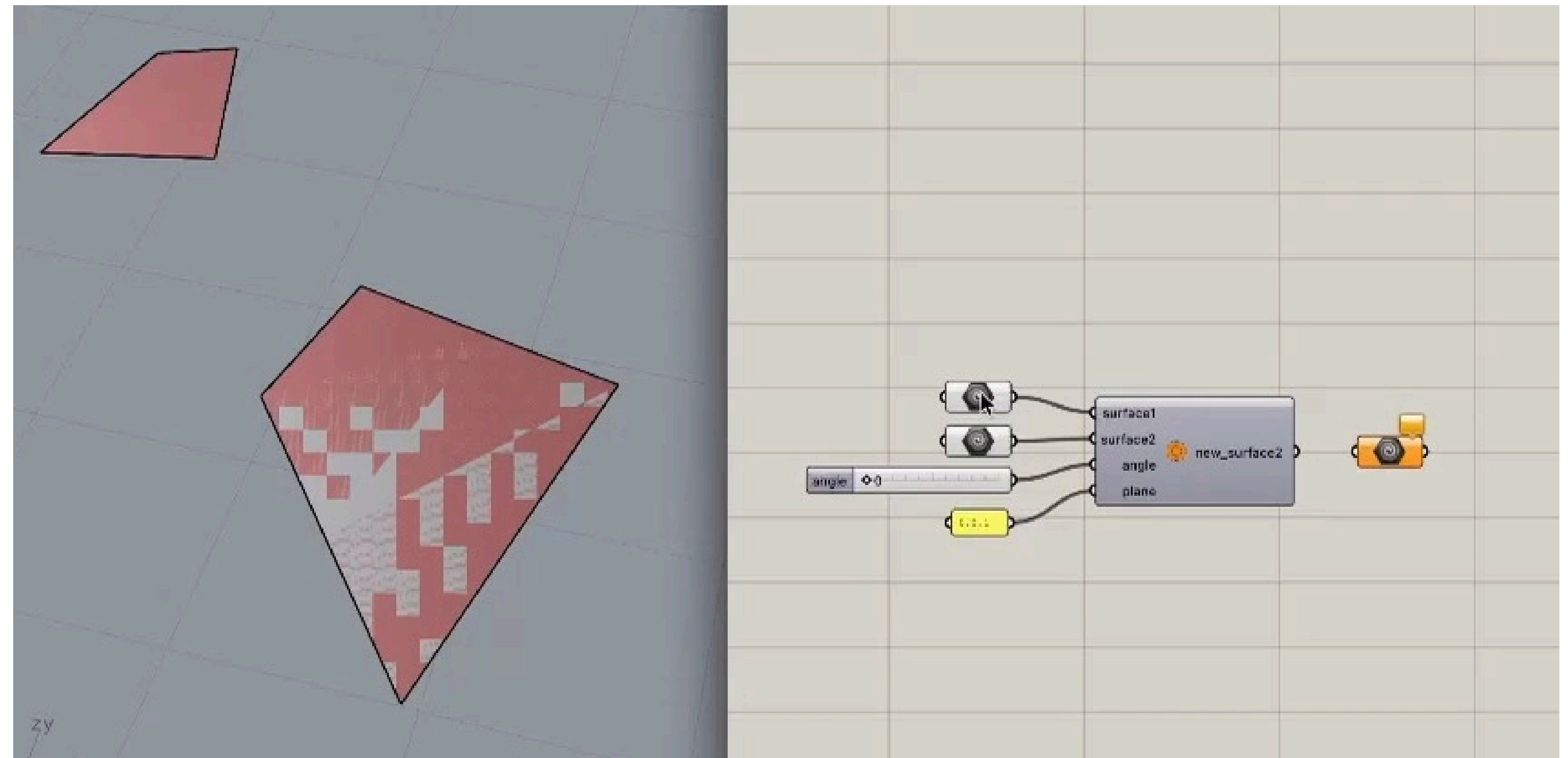
surface2: target surface

angle: angle number

plane: string "0, 0, 1"

OUTPUT:

new_surface2: surface2 that rotate via surface1 midpoint



Get Midpoint

Input geometry to get the midpoint

INPUT:

obj:

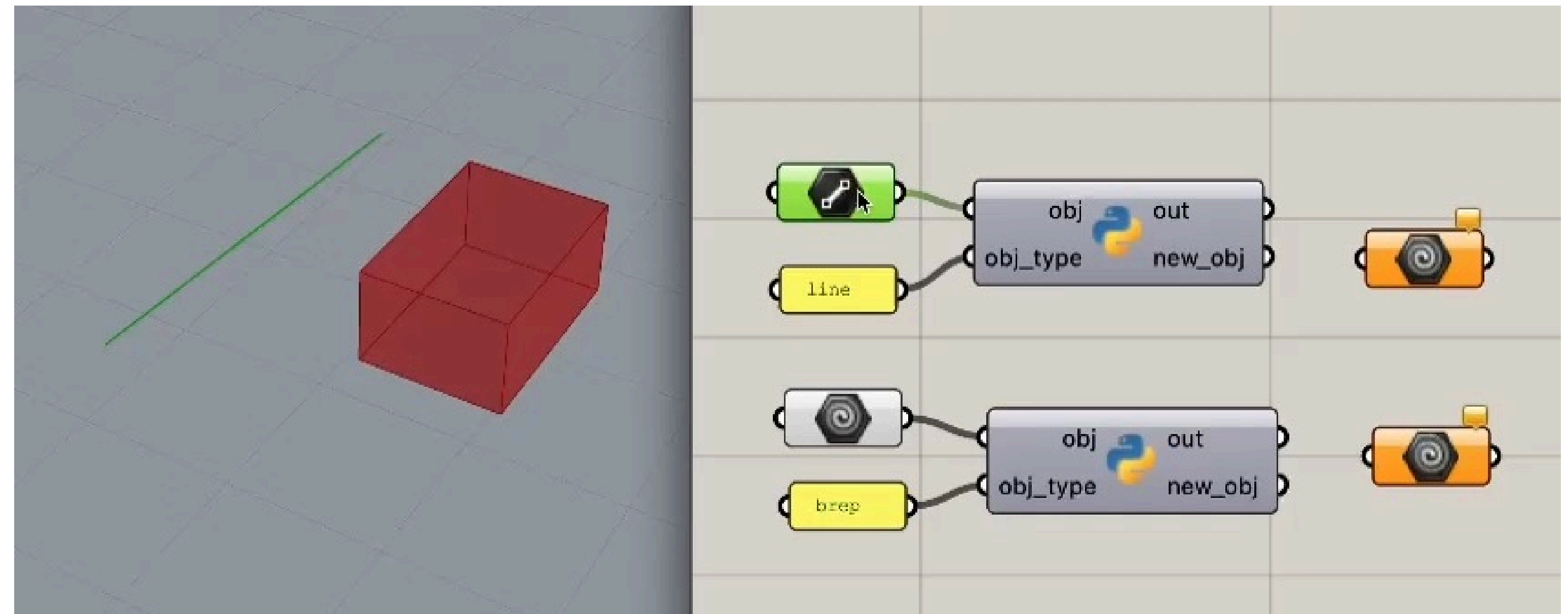
line, surface, mesh or brep

obj_type:

string - "line", "surface",
"mesh" or "brep"

OUTPUT:

new_obj: midpoint



Get Parallel

Let input obj1 to parallel with obj2

INPUT:

obj1:

reference LineCurve, Surface, Brep

obj2:

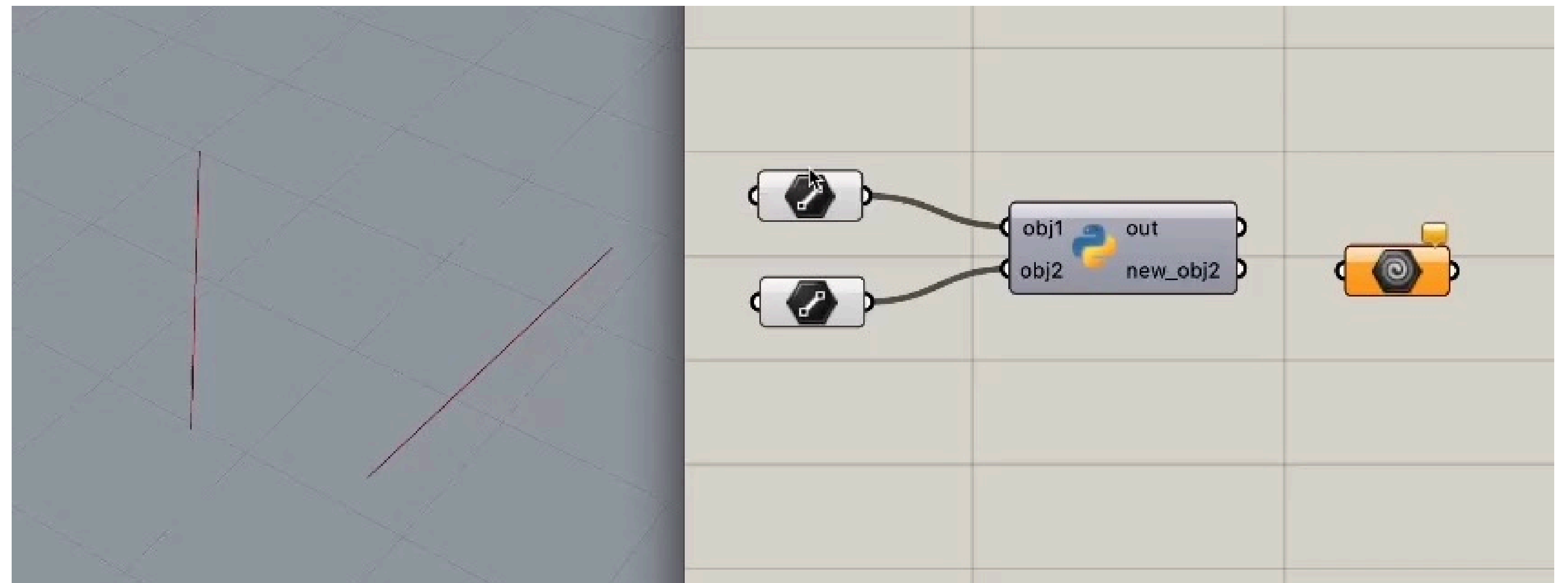
target LineCurve, Surface, Brep

OUTPUT:

new_obj2:

obj2 that parallel with obj1

obj1 and obj2 need to be same type



Get Perpendicular

Let input obj1 to perpendicular with obj2

obj1 and obj2 need to be same type

INPUT:

obj1:

reference LineCurve, Surface, Brep

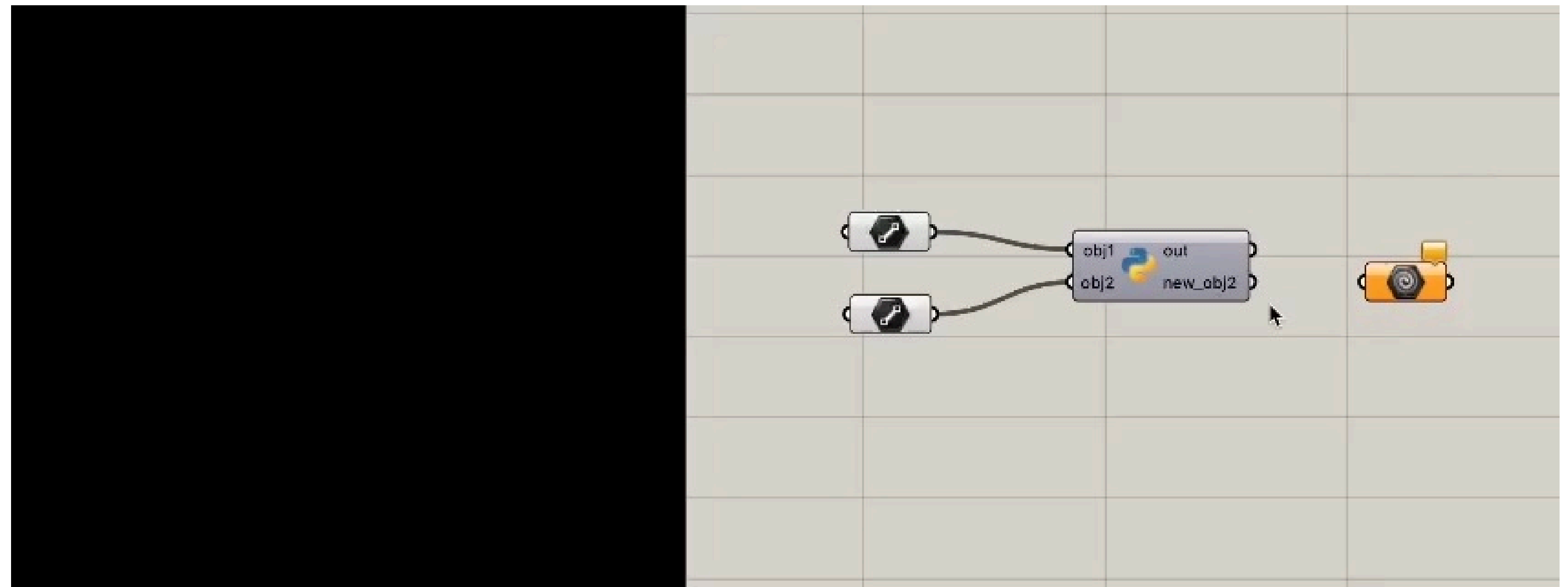
obj2:

target LineCurve, Surface, Brep

OUTPUT:

new_obj2:

obj2 that perpendicular with obj1



Move Tools

We want to develop a set of move tools that enable objects to move or rotate in the intended path

- Line Mover
- Object Rotator

Line Mover

Let one object to move along a path

INPUT:

obj: target Brep

path: PolyLineCurve

steps: number - 0 to 100

OUTPUT:

new_obj:

new brep that move along path



Object Rotator

Let Obj1 rotate coaxially with Obj2

INPUT:

obj1: Brep

obj2: Brep

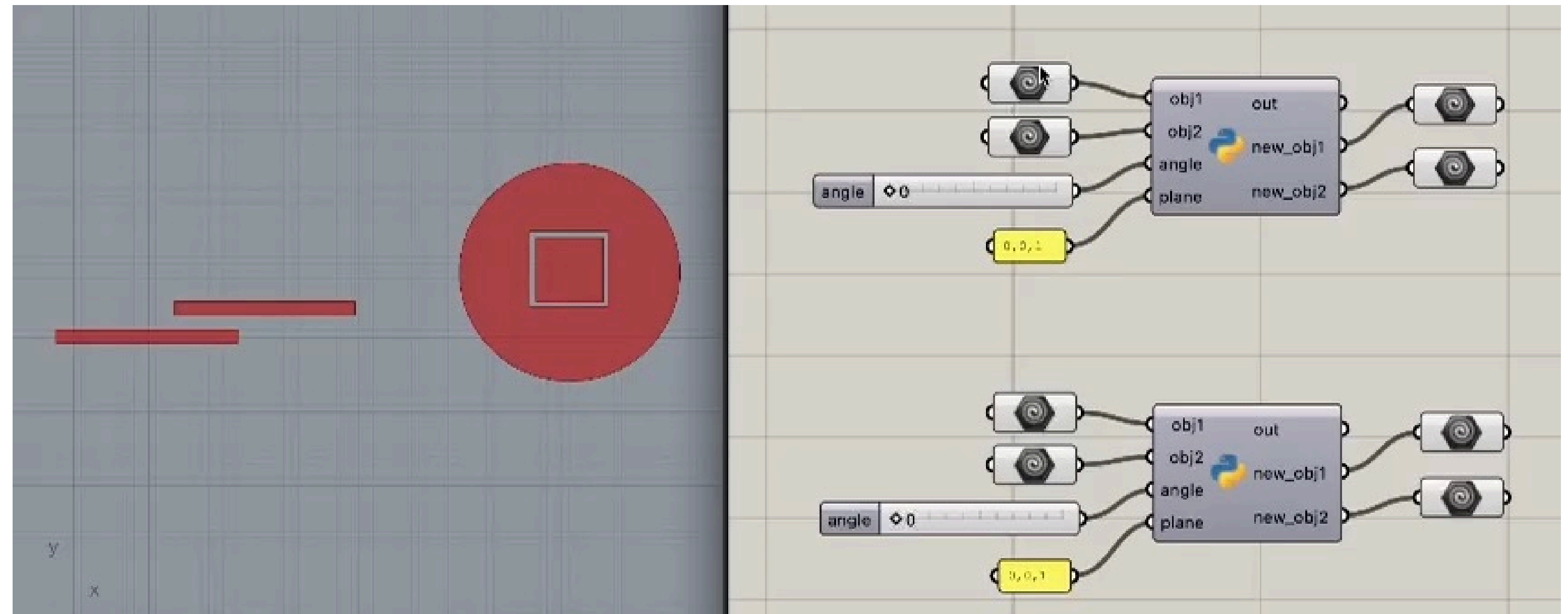
angle: angle number

plane: string "0, 0, 1"

OUTPUT:

new_obj1: Brep

new_obj2: Brep



Publish

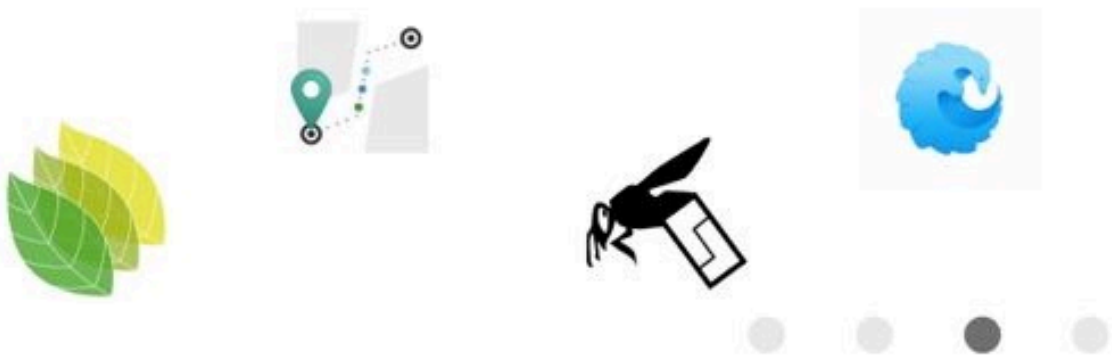


+

 filters

Search

food4Rhino webinars playlist



ASSEMBLER (by Julia_93)

!

This content is not yet published.



This is a set of components that automates the process of composing these elements efficiently, reduces the time spent aligning parts and ensure precision in assembling these elements.

Edit

Download



Downloads:

Support Email

Category: Product design

License Type: MIT

Cost: Free

Conclusion



Function Name	Input	Output
Point Alignment	point1 (Rhino Point) point2 (Rhino Point) obj (Rhino GeometryBase)	new_obj (Rhino GeometryBase)
Line Alignment	line1 (Rhino LineCurve) line2 (Rhino LineCurve) x, y, z (Number)	aligned_line (Rhino LineCurve)
Surface Alignment	surface1 (Rhino Surface) surface2 (Rhino Surface) x, y, z (Number)	new_surface2 (Rhino Surface)
Center Alignment	surface1 (Rhino Surface) surface2 (Rhino Surface) angle (Number) plane (Str)	new_surface2 (Rhino Surface)

Get Midpoint	obj (Rhino Line, Surface, Brep, Mesh) obj_type (Str [Line, Surface, Brep, Mesh])	new_obj (Rhino Point)
Get parallel	obj1, obj2 (Rhino [LineCurve, LineCurve Surface, Surface Brep, Brep])	new_obj2 (Rhino Line, Surface, Brep)
Get perpendicular	obj1, obj2 (Rhino [LineCurve, LineCurve Surface, Surface Brep, Brep])	new_obj2 (Rhino Line, Surface, Brep)

Function Name	Input	Output
Line Mover	obj (Rhino Brep) path (Rhino PolyLineCurve) steps (Number [0~100])	new_obj (Rhino Brep)
Object Rotator	obj1 (Rhino Brep) obj2 (Rhino Brep) angle (Number) plane (Str)	new_obj1 (Rhino Brep) new_obj2 (Rhino Brep)

Function Name	Input	Output
Object Selection	obj (Rhino PolylineCurve, Brep, Mesh) index (Number) select_type (Str [point, line, surface])	target_obj (Rhino Point, PolylineCurve, Surface)
Object Collision	g1 (Rhino Mesh) g2 (Rhino Mesh)	is_interecting (Str [True, False])
Single Point Contact	g1 (Rhino Point, Line, PolyLine, Curve, Brep) g2 (Rhino Point, Line, PolyLine, Curve, Brep)	is_interecting (Str [True, False])